Информационные технологии и программирование

3 семестр
Лекция 9.

Содержание лекции:

- Avalonia
- Начало работы
- Reactive UI
- Программа на C# Avalonia + Reactive UI + MVVM

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Avalonia

Avalonia — это кроссплатформенный фреймворк для построения графических пользовательских интерфейсов (GUI) на основе XAML. Разработана на основе WPF/UWP и распространяемая под лицензией MIT. Ранее называлась Perspex.

Полностью поддерживает .NET Core 2.0, XAML, дата-биндинги, lookless-контролы и многое другое. Avalonia позволяет писать на C# приложения под Windows, Linux и macOS.

Начиная с Avalonia 11 preview 1 появилась возможность запуска на iOS, Android и Web (без Blazor) в экспериментальном состоянии.

Год появления: ~2013 (первый коммит), стабильное и широкое признание получил с выходом .NET Core и .NET 5+.

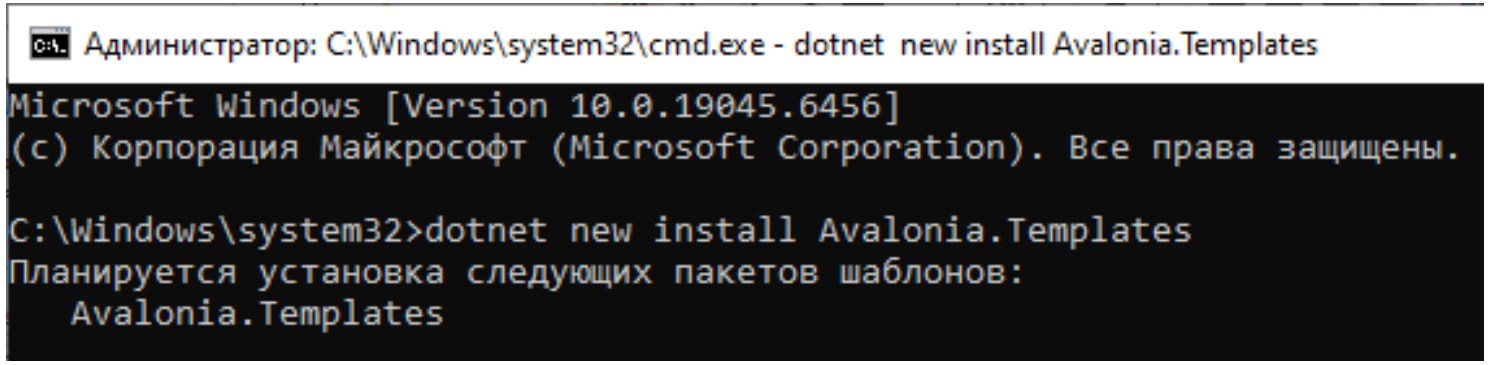
Кроссплатформенность – свойство программного обеспечения, позволяющее ему работать с несколькими операционными системами или аппаратными платформами без изменения исходного кода.

Привязка данных (Data Binding) – механизм автоматической синхронизации данных между пользовательским интерфейсом (View) и источником данных (ViewModel, Model). При изменении данных в коде интерфейс автоматически обновляется, и наоборот.

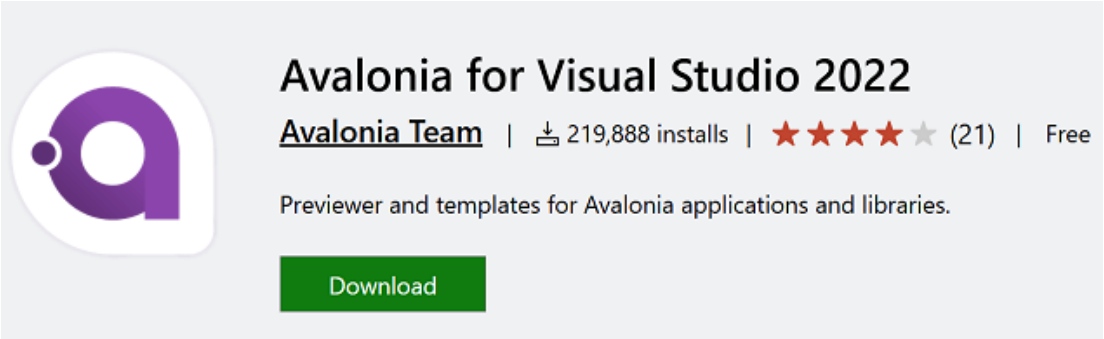
Начало работы

Для работы с Avalonia необходимо установить шаблоны, для этого нужно запустить консоль и выполнить команду:

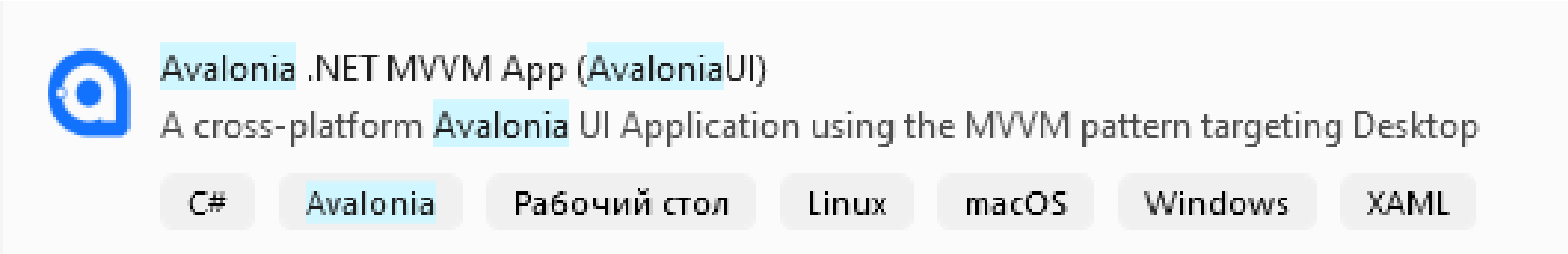
dotnet new install Avalonia.Templates



Для поддержки Avalonia в Visual Studio, необходимо установить расширение Avalonia для Visual Studio.



Чтобы создать проект в рамках шаблона MVVM необходимо выбрать



Остальные пункты прокликиваются далее.

Структура проекта:

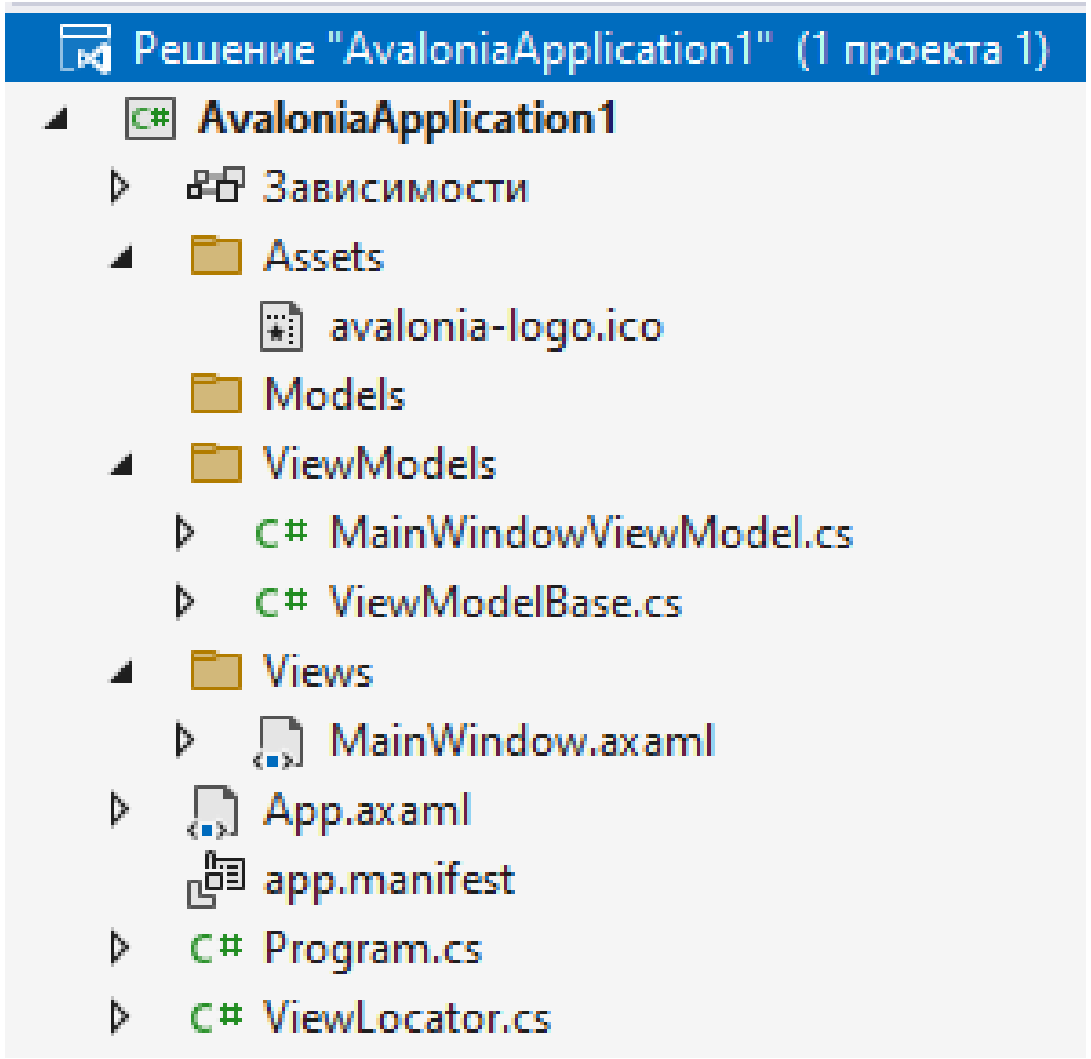
Assets – статические файлы ресурсов (иконки, стили и др.)

Models, ViewModels, Views – модели, модели представлений и представления согласно паттерну MVVM.

App – точка для запуска основного окна (MainWindow).

Program – точка входа в программу.

ViewLocator – вспомогательный класс, автоматически связывает Views и ViewModels по именам.

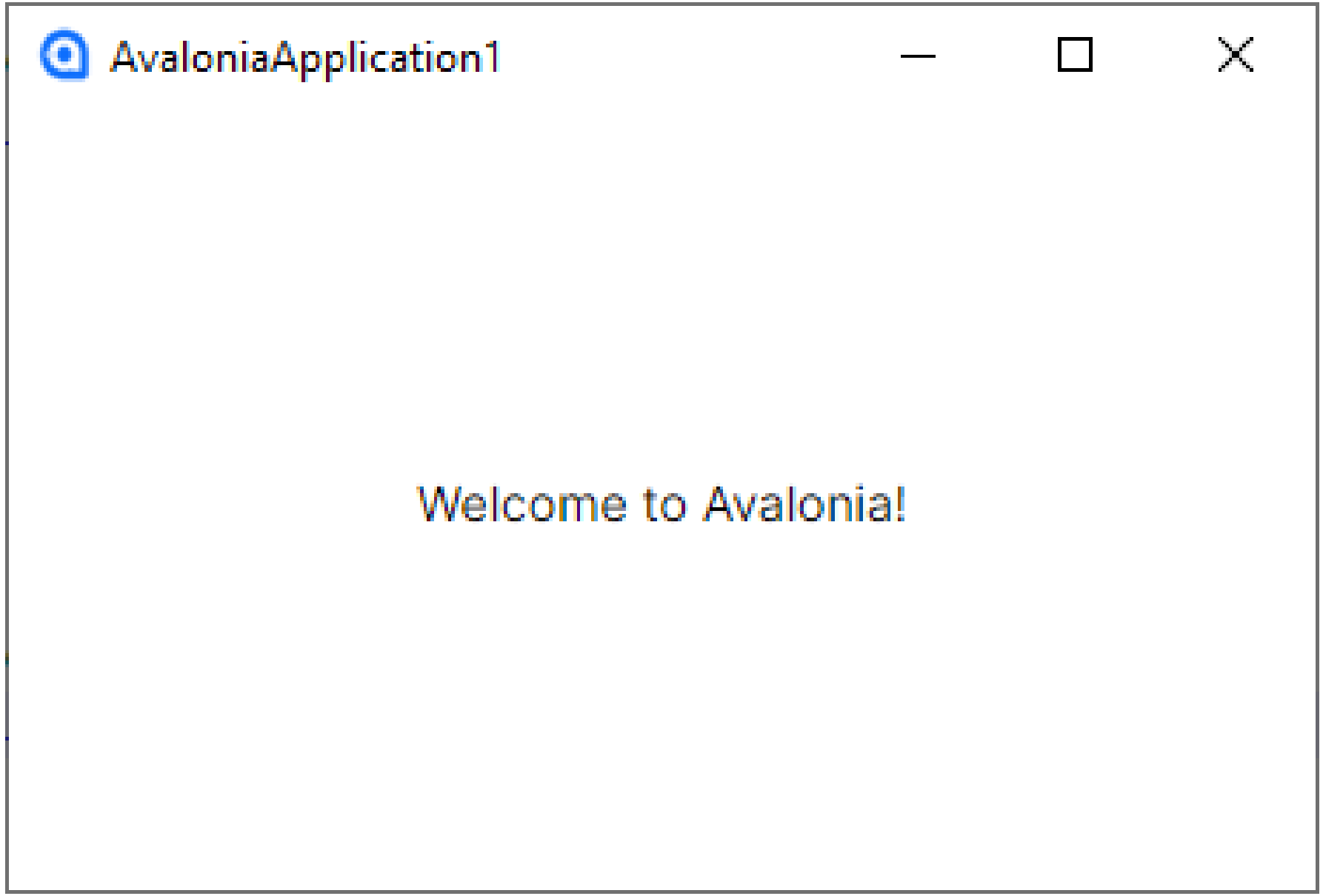


Базовый проект включает следующую модель представления MainWindowViewModel.cs:

```
namespace AvaloniaApplication1.ViewModels;

public partial class MainWindowViewModel : ViewModelBase
{
    public string Greeting { get; } = "Welcome to Avalonia!";
}
```

Представление MainWindow.xaml:

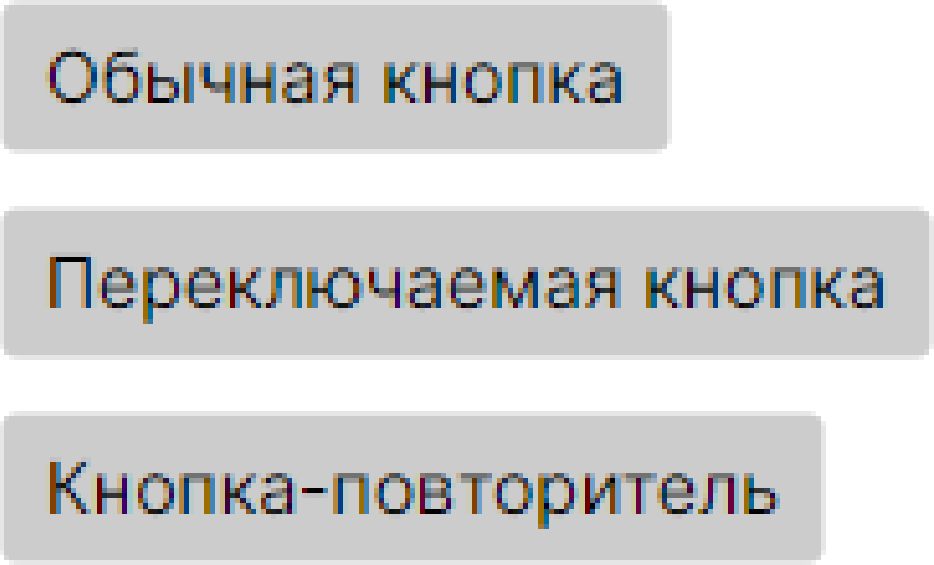
```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:vm="using:AvaloniaApplication1.ViewModels"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" d:DesignWidth="800" d:DesignHeight="450"
        x:Class="AvaloniaApplication1.Views.MainWindow"
        x:DataType="vm:MainWindowViewModel"
        Icon="/Assets/avalonia-logo.ico"
        Title="AvaloniaApplication1">

    <Design.DataContext>
        <vm:MainWindowViewModel/>
    </Design.DataContext>

    <TextBlock Text="{Binding Greeting}" HorizontalAlignment="Center"
VerticalAlignment="Center"/>

</Window>
```


AvaloniaApplication1
Welcome to Avalonia!

Базовые элементы управления

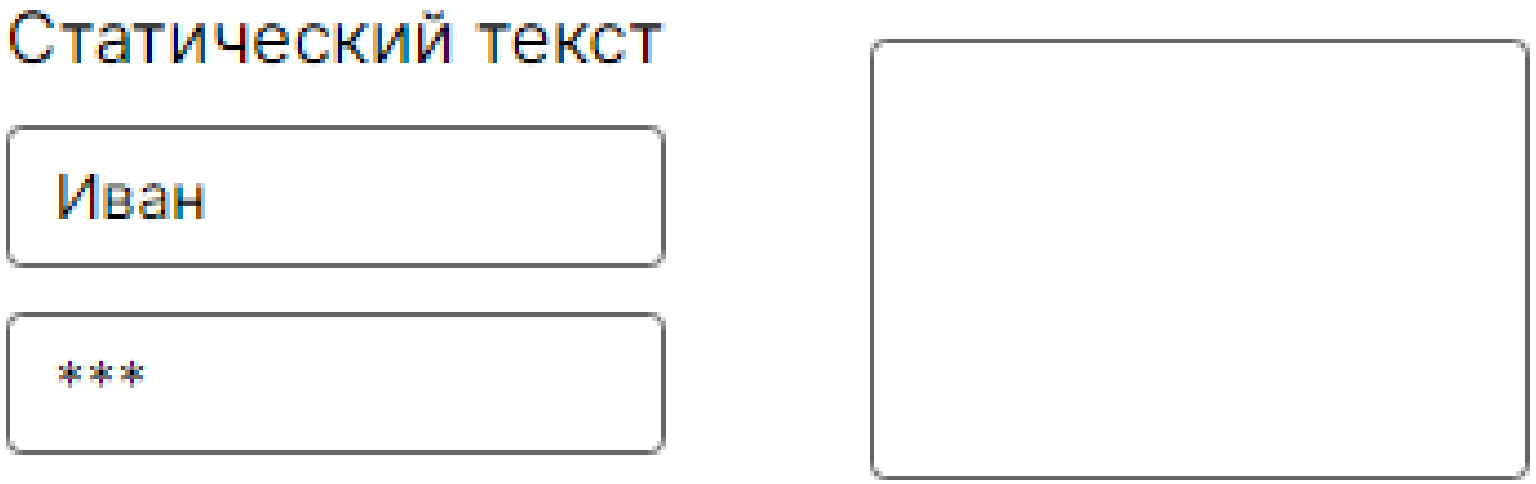
1. Кнопки

```
<Button Content="Обычная кнопка" Click="Button_Click"/>
<ToggleButton Content="Переключаемая кнопка"/>
<RepeatButton Content="Кнопка-повторитель“
              Delay="500" Interval="100"/>
```



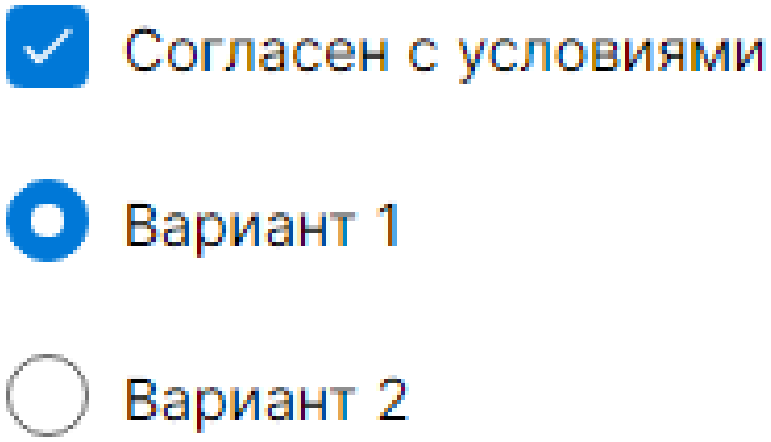
2. Текстовые элементы

```
<!-- Простой текст -->
<TextBlock Text="Статический текст" FontSize="16"/>
<!-- Поле ввода -->
<TextBox Text="{Binding Name}" Watermark="Введите имя"/>
<!-- Пароль -->
<TextBox PasswordChar="*" Text="{Binding Password}"/>
<!-- Многострочный текст -->
<TextBox AcceptsReturn="True" TextWrapping="Wrap" Height="100"/>
```



3. Элементы выбора

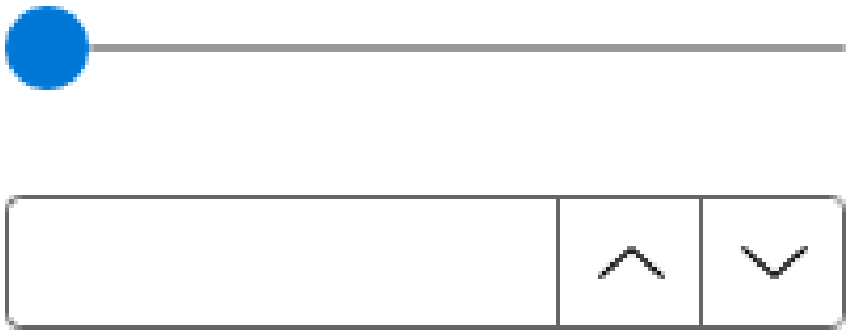
```
<!-- Чекбокс -->
<CheckBox Content="Согласен с условиями"
          IsChecked="{Binding IsAgreed}"/>
<!-- Радио-кнопки -->
<RadioButton GroupName="Group1"
             Content="Вариант 1" IsChecked="{Binding Option1}"/>

<RadioButton GroupName="Group1"
             Content="Вариант 2" IsChecked="{Binding Option2}"/>
```



```
<!-- Выпадающий список -->
<ComboBox ItemsSource="{Binding Items}"
          SelectedItem="{Binding SelectedItem}">

    <ComboBox.ItemTemplate>
        <DataTemplate>
            <TextBlock Text="{Binding Name}"/>
        </DataTemplate>
    </ComboBox.ItemTemplate>

</ComboBox>

<!-- Список -->
<ListBox ItemsSource="{Binding Users}"
         SelectedItem="{Binding SelectedUser}"/>
```

4. Элементы для работы с числами

```
<!-- Ползунок -->
<Slider Minimum="0" Maximum="100" Value="{Binding Volume}"/>

<!-- Счетчик -->
<NumericUpDown Value="{Binding Age}" Minimum="0" Maximum="120"/>
```



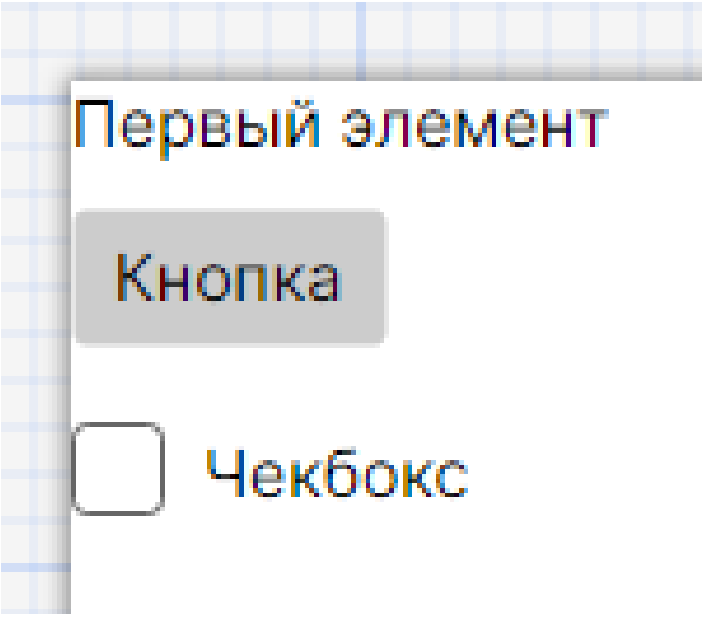
5. Изображения

```
<Image Source="image.png" Width="100" Height="100"/>
<Image Source="{Binding ImagePath}"/>
```

Контейнеры для компоновки

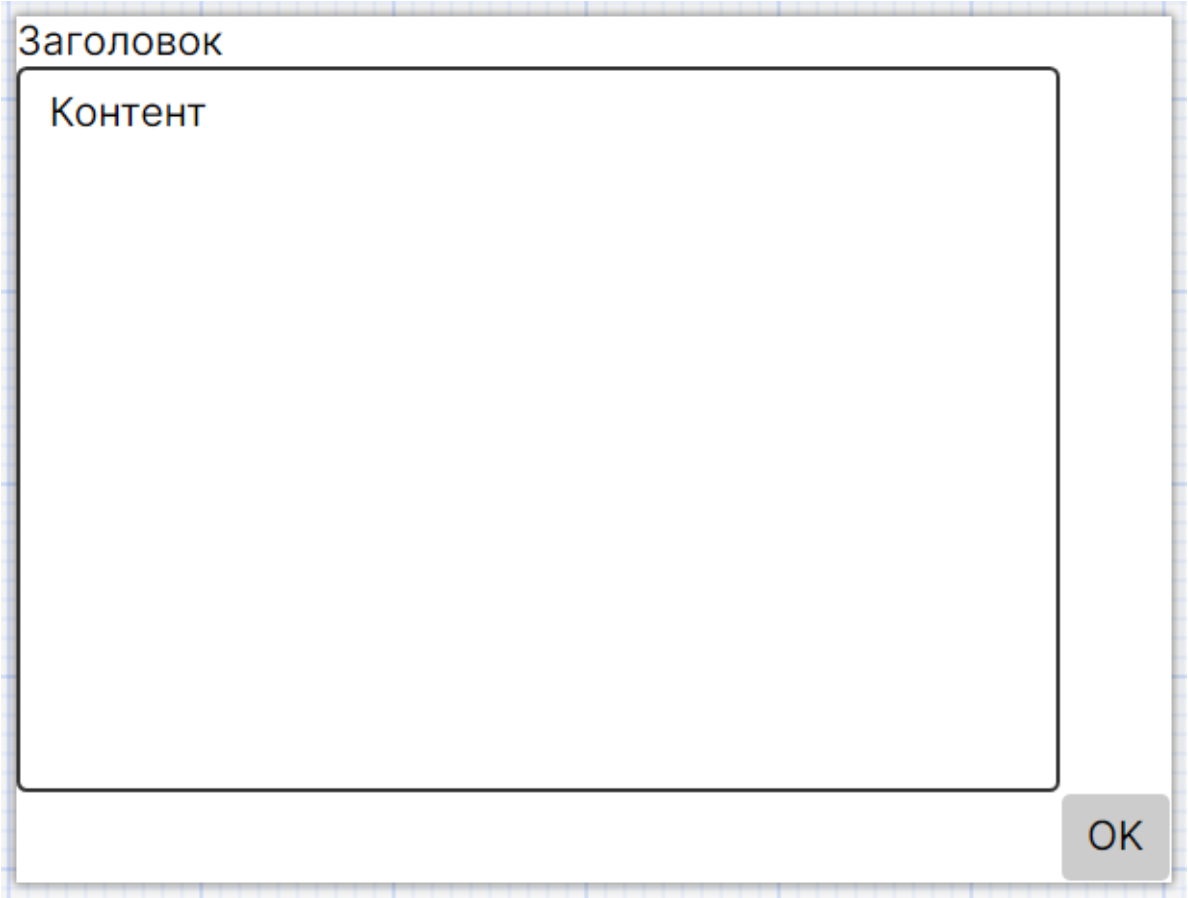
1. StackPanel - линейное расположение

```
<StackPanel Orientation="Vertical" Spacing="10">
    <TextBlock Text="Первый элемент"/>
    <Button Content="Кнопка"/>
    <CheckBox Content="Чекбокс"/>
</StackPanel>
```



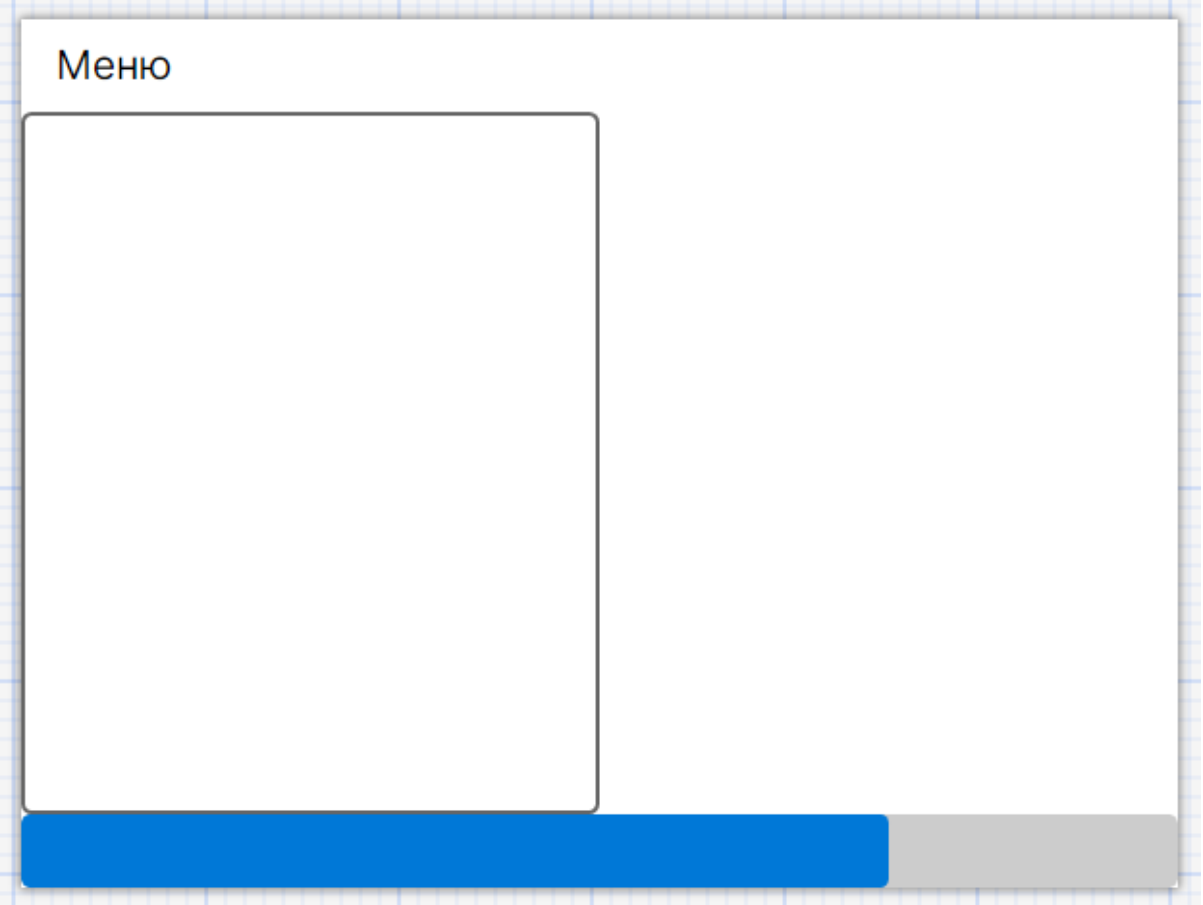
2. Grid - табличное расположение

```
<Grid ColumnDefinitions="100,*,Auto" RowDefinitions="Auto,*,Auto">
    <TextBlock Grid.Row="0" Grid.Column="0" Text="Заголовок"/>
    <TextBox Grid.Row="1" Grid.Column="0" Grid.ColumnSpan="2"
Text="Контент"/>
    <Button Grid.Row="2" Grid.Column="2" Content="OK"/>
</Grid>
```



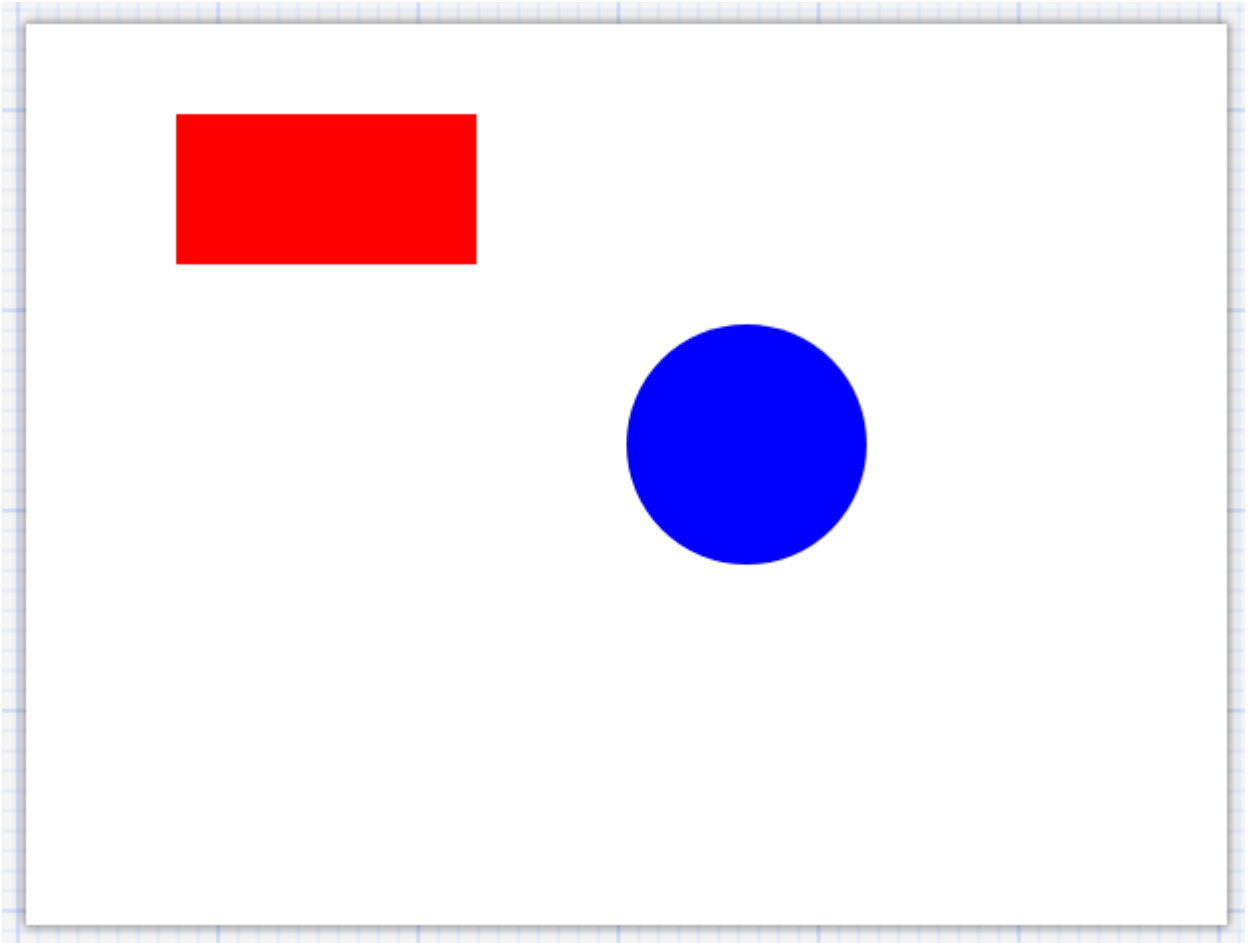
3. DockPanel - прикрепление к краям

```
<DockPanel>
    <Menu DockPanel.Dock="Top">Меню</Menu>
    <ProgressBar Value="75" DockPanel.Dock="Bottom"
Height="25"></ProgressBar>
    <TextBox DockPanel.Dock="Left" Width="200"/>
    <ContentControl Content=""/>
</DockPanel>
```



4. Canvas - абсолютное позиционирование

```
<Canvas>
    <Rectangle Canvas.Left="50" Canvas.Top="30" Width="100"
Height="50" Fill="Red"/>
    <Ellipse Canvas.Left="200" Canvas.Top="100" Width="80"
Height="80" Fill="Blue"/>
</Canvas>
```



Специальные элементы

1. StackPanel - линейное расположение

```
<DataGrid ItemsSource="{Binding Users}" AutoGenerateColumns="False">
    <DataGrid.Columns>
        <DataGridTextColumn Header="Имя" Binding="{Binding Name}"/>
        <DataGridCheckBoxColumn Header="Активен" Binding="{Binding IsActive}"/>
    </DataGrid.Columns>
</DataGrid>
```

2. TreeView - дерево элементов

```
<TreeView ItemsSource="{Binding Categories}">
    <TreeView.ItemTemplate>
        <TreeDataTemplate ItemsSource="{Binding SubCategories}">
            <TextBlock Text="{Binding Name}"/>
        </TreeDataTemplate>
    </TreeView.ItemTemplate>
</TreeView>
```

3. TabControl - вкладки

```
<TabControl>
    <TabItem Header="Вкладка 1">
        <TextBlock Text="Содержимое первой вкладки"/>
    </TabItem>
    <TabItem Header="Вкладка 2">
        <Button Content="Кнопка на второй вкладке"/>
    </TabItem>
</TabControl>
```

4. Menu и ContextMenu

```
<!-- Главное меню -->
<Menu>
    <MenuItem Header="Файл">
        <MenuItem Header="Открыть" Command="{Binding OpenCommand}"/>
        <MenuItem Header="Сохранить" Command="{Binding
SaveCommand}"/>
        <Separator/>
        <MenuItem Header="Выход" Command="{Binding ExitCommand}"/>
    </MenuItem>
</Menu>

<!-- Контекстное меню -->
<TextBox ContextMenu="{StaticResource TextContextMenu}"/>
```

5. ScrollViewer - прокрутка

```
<ScrollViewer HorizontalScrollBarVisibility="Auto"
VerticalScrollBarVisibility="Auto">
    <StackPanel Height="1000">
        <!-- Длинный контент -->
    </StackPanel>
</ScrollViewer>
```

Reactive UI

Reactive UI (RxUI) — это фреймворк, построенный на основе реактивного программирования (Reactive Extensions, или Rx), который позволяет создавать приложения, реагирующие на изменения данных, используя наблюдаемые последовательности (Observables).

Зачем он нужен?

Упрощение MVVM: Избавляет от рутинного кода (вызовов INotifyPropertyChanged, команд).

Мощные привязки: Позволяет привязывать не только свойства, но и целые потоки событий.

Управление состоянием: Упрощает обработку асинхронных операций, валидации и сложных взаимодействий между компонентами.

Основные Термины и Концепции

1. ReactiveObject

Базовый класс для всех ViewModel, который реализует INotifyPropertyChanged реактивным способом.

```
public class MainViewModel : ReactiveObject
{
    private string _name;
    public string Name
    {
        get => _name;
        set => this.RaiseAndSetIfChanged(ref _name, value);
    }
}
```

RaiseAndSetIfChanged — это метод, который проверяет, изменилось ли значение, и уведомляет об этом UI.

2. Привязки (Bindings) через WhenActivated и OneWayBind, Bind

В Avalonia Reactive UI привязки часто настраиваются в конструкторе View с помощью метода WhenActivated.

```
public partial class MainWindow : ReactiveUserControl<MainViewModel>
{
    public MainWindow()
    {
        InitializeComponent();
        // Когда View становится активной, устанавливаются привязки
        this.WhenActivated(disposables =>
        {
            // OneWayBind: Односторонняя привязка (из VM в View)
            this.OneWayBind(ViewModel,
                            vm => vm.Name, v => v.NameTextBlock.Text)
                .DisposeWith(disposables);

            // Bind: Двусторонняя привязка (например, для TextBox)
            this.Bind(ViewModel, vm => vm.Name, v => v.NameTextBox.Text)
                .DisposeWith(disposables);
        });
```

3. Команды (ReactiveCommand)

Команда, которая может быть включена или отключена на основе наблюдаемой последовательности (Observable).

```
public partial class MainWindowViewModel : ViewModelBase
{

// Простая команда без параметра
public ReactiveCommand<Unit, Unit> SubmitCommand { get; }

// Команда с параметром string
public ReactiveCommand<string, Unit> LoadDataCommand { get; }

}
```

```
public MainWindowViewModel()
{
    // CanExecute: определяет, когда команда активна.
    var canSubmit = this.WhenAnyValue(vm => vm.Name,
                                      name => !string.IsNullOrWhiteSpace(name));

    SubmitCommand = ReactiveCommand.Create(ExecuteSubmit, canSubmit);

    // Команда с параметром
    LoadDataCommand = ReactiveCommand.Create<string>(ExecuteLoadData);
}
private void ExecuteSubmit()
{
    File.WriteAllText ("data.txt", Name);
}
private void ExecuteLoadData(string parameter)
{
    // Логика загрузки данных по параметру
    Name = $"Loaded: {parameter}";
}
```

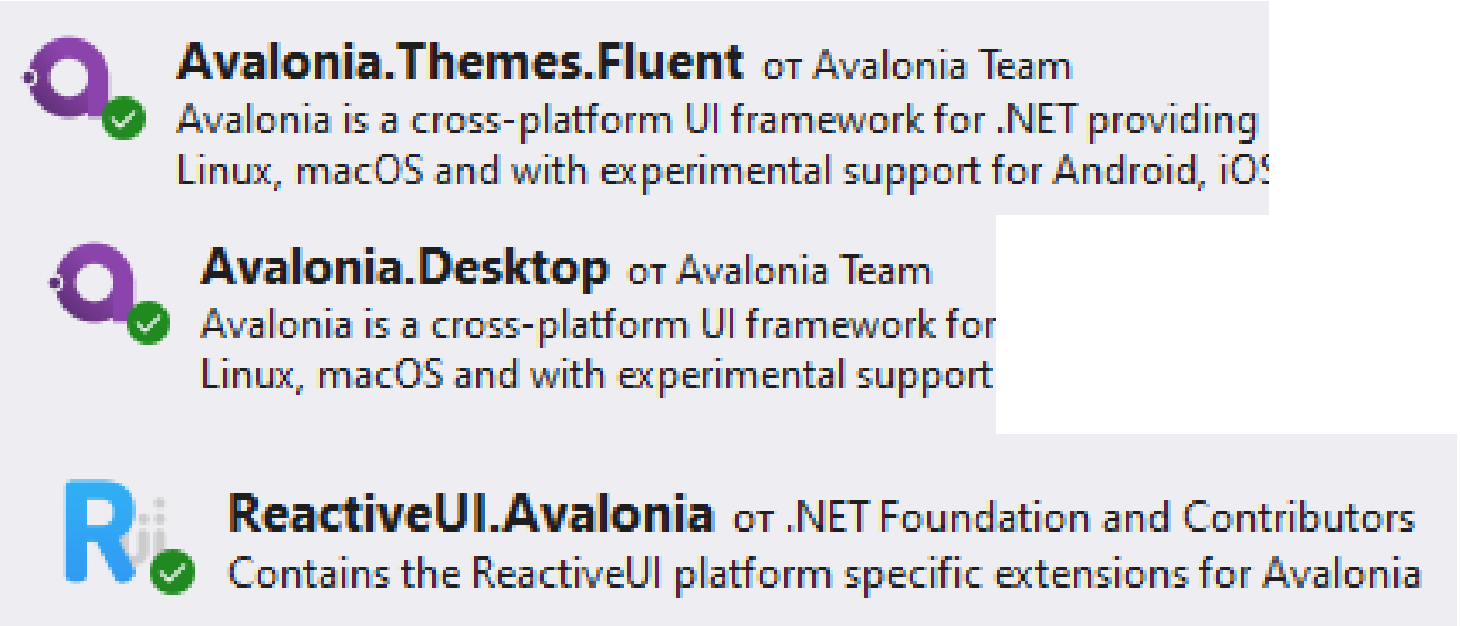
Программа на C# Avalonia + Reactive UI + MVVM

Структура проекта с ReactiveUI

```
ReactiveAvaloniaApp/
├── ViewModels/
│   └── MainWindowViewModel.cs
├── Views/
│   └── MainWindow.axaml
│   └── MainWindow.axaml.cs
├── App.axaml
├── App.axaml.cs
└── Program.cs
```

1. Установка необходимых пакетов

```
<ItemGroup>
    <PackageReference Include="Avalonia.Desktop" Version="11.0.5" />
    <PackageReference Include="Avalonia.Themes.Fluent" Version="11.0.5" />
    <PackageReference Include="ReactiveUI.Avalonia" Version="19.4.1" />
</ItemGroup>
```



В Program.cs необходимо добавить строку `UseReactiveUI()`

```
public static AppBuilder BuildAvaloniaApp()
            => AppBuilder.Configure<App>()
                .UsePlatformDetect()
                .WithInterFont()
                .LogToTrace()
                .UseReactiveUI(); //<-----
```

2. ViewModel с ReactiveUI

ViewModels/MainWindowViewModel.cs

```
public class MainWindowViewModel : ReactiveObject
{
    private string _message = "Добро пожаловать в Avalonia + ReactiveUI!";
    private int _count;

    public MainWindowViewModel()
    {
         // Создаем вычисляемое свойство, которое реагирует на изменения Count
        this.WhenAnyValue(x => x.Count)
            .Select(count => $"Количество нажатий: {count}")
            .ToProperty(this, x => x.DisplayCount, out _displayCount);

        // Инициализация команд
        IncrementCommand = ReactiveCommand.Create(IncrementCount);

        // Команда сброса активна только когда счетчик > 0
        ResetCommand = ReactiveCommand.Create(ResetCount,
            this.WhenAnyValue(x => x.Count).Select(count => count > 0));
    }
```

```
// Reactive свойства
public string Message
{
    get => _message;
    set => this.RaiseAndSetIfChanged(ref _message, value);
}

public int Count
{
    get => _count;
    set => this.RaiseAndSetIfChanged(ref _count, value);
}

// Вычисляемое свойство через Observable
private readonly ObservableAsPropertyHelper<string> _displayCount;
public string DisplayCount =>
                _displayCount?.Value ?? "Количество нажатий: 0";
```

```
    // Команды
    public ReactiveCommand<Unit, Unit> IncrementCommand { get; }
    public ReactiveCommand<Unit, Unit> ResetCommand { get; }

    private void IncrementCount()
    {
        Count++;

        if (Count % 10 == 0)
            Message = $"Отлично! Вы достигли {Count} нажатий!";
    }

    private void ResetCount()
    {
        Count = 0;
        Message = "Счетчик сброшен! Давайте начнем снова.";
    }
}
```

3. View с ReactiveUI

Views/MainWindow.axaml

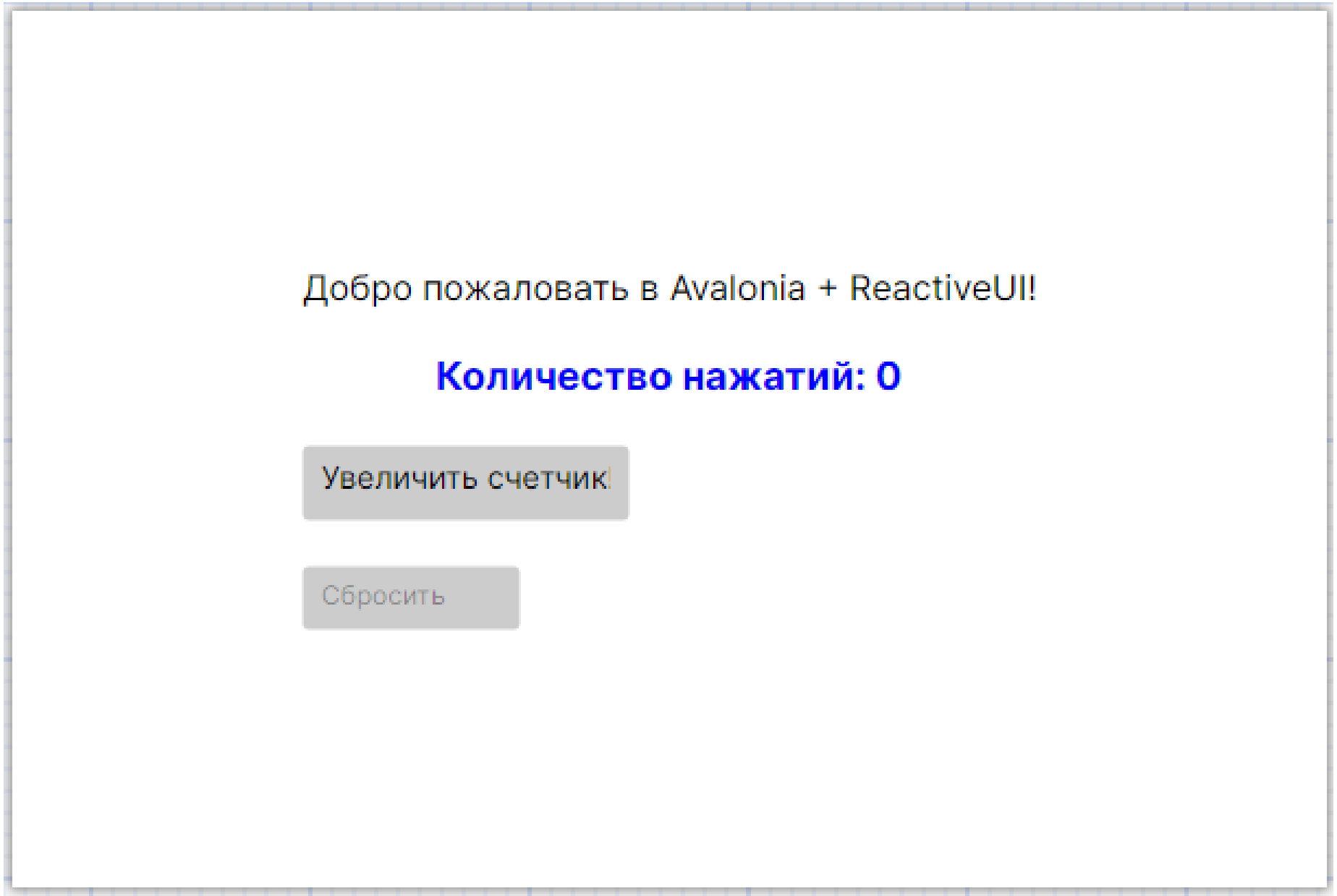
```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:vm="using:AvaloniaApplication1.ViewModels"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" Width="600" Height="400"
        x:Class="AvaloniaApplication1.Views.MainWindow"
        x:DataType="vm:MainWindowViewModel"
        Icon="/Assets/avalonia-logo.ico"
        Title="Первое приложение на Avalonia">

    <Design.DataContext>
        <vm:MainWindowViewModel/>
     </Design.DataContext>

        <!– Далее -->

</Window>
```

```
<StackPanel HorizontalAlignment="Center"
            VerticalAlignment="Center" Spacing="20">

    <!-- Приветственный текст -->
    <TextBlock Text="{Binding Message}"
               FontSize="16"
               TextAlignment="Center"
               HorizontalAlignment="Center"/>

    <!-- Счетчик с привязкой к вычисляемому свойству -->
    <TextBlock Text="{Binding DisplayCount}"
               FontSize="18"
               FontWeight="Bold"
               Foreground="Blue"
               HorizontalAlignment="Center"/>
```

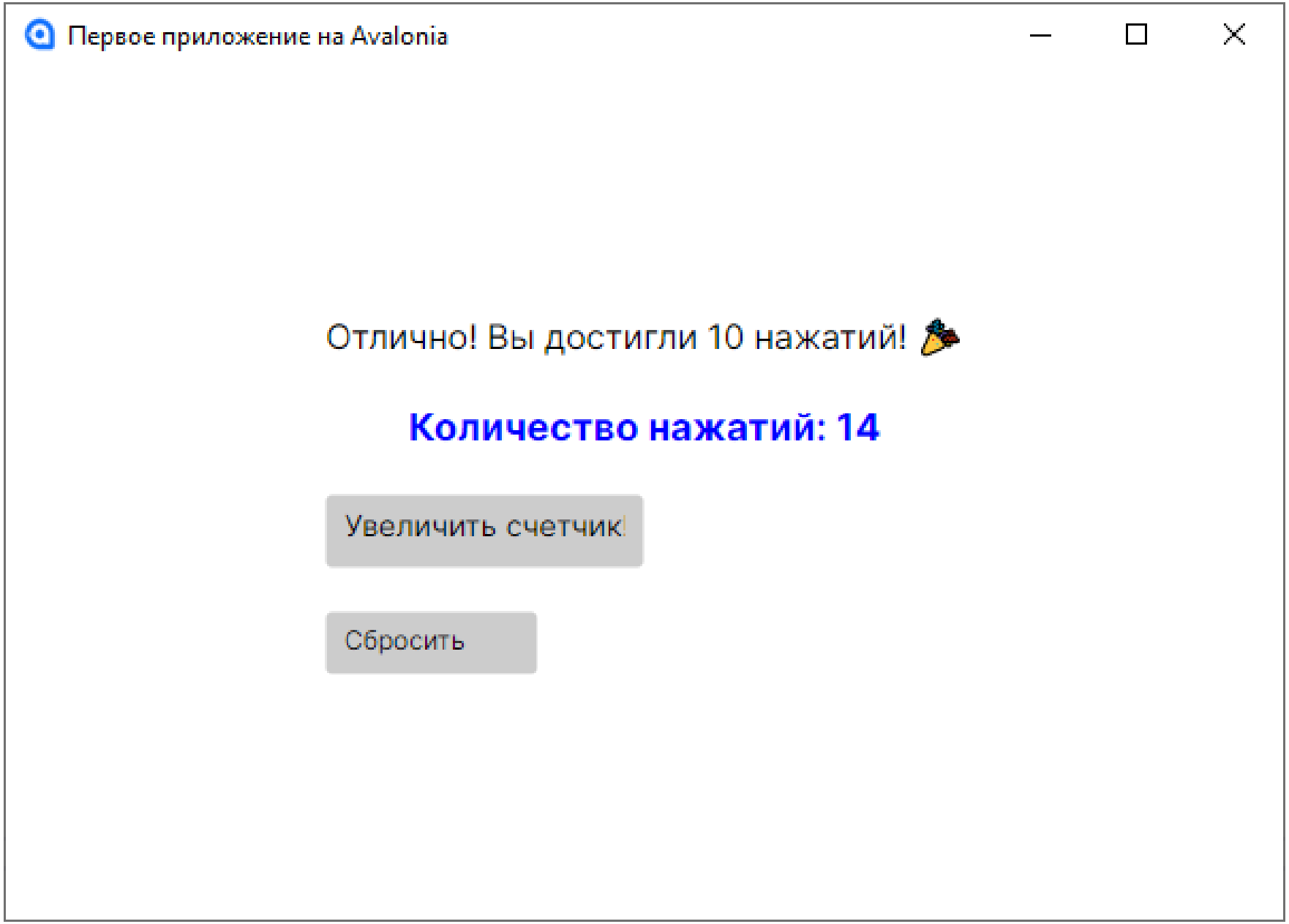
```
        <!-- Кнопки с привязкой команд ReactiveUI -->
        <Button Content="Увеличить счетчик!"
                Command="{Binding IncrementCommand}"
                Width="150" Height="35"
                FontSize="14"/>

        <Button Content="Сбросить"
                Command="{Binding ResetCommand}"
                Width="100" Height="30"
                FontSize="12"/>

    </StackPanel>
```

Замечание: Чтобы отображалась кириллица необходимо, чтобы расширение файла было UTF-8.

Внешний вид окна



Пример работы программы



Открытие второстепенного окна

Создадим команду для открытия окна.

Добавим в конструктор строку:

```
WindowCommand = ReactiveCommand.Create(OpenWindow);
```

Добавим свойство:

```
public ReactiveCommand<Unit, Unit> WindowCommand { get; }
```

Метод открытия окна:

```
private void OpenWindow()
{
    var vm = new SecondWindowViewModel();

    var view = new SecondWindow
    {
        DataContext = vm
    };

    var parent = Avalonia.Application.Current?.ApplicationLifetime
                    as IClassicDesktopStyleApplicationLifetime;

    if (parent == null || parent.MainWindow == null)
        throw new InvalidOperationException(
              "Невозможно определить родительское окно");

    view.ShowDialog(parent.MainWindow);
}
```